



README – PaintKiller

Primera Entrega – Prototipo Inicial

1. Descripción General

En esta primera entrega he desarrollado la base estructural del proyecto *PaintKiller*, un videojuego 2D creado en Unity.

El objetivo principal de esta fase no ha sido completar el juego final, sino establecer una base técnica sólida y coherente que permita desarrollar las mecánicas futuras con estabilidad y orden.

Durante esta entrega he realizado:

- Creación del proyecto desde cero en Unity.
- Organización estructurada de carpetas siguiendo una arquitectura modular.
- Construcción del escenario base.
- Implementación del sistema de movimiento del Runner.
- Configuración del sistema físico y colisiones.
- Integración inicial de una mecánica asimétrica básica (drag del Runner).

He priorizado la estabilidad, claridad estructural y separación de responsabilidades frente a la cantidad de funcionalidades.

2. Organización del Proyecto

El proyecto se ha estructurado de forma modular para facilitar su escalabilidad futura.

Se han separado claramente las siguientes capas:

- **Input** → Captura de entrada del jugador.
- **Logic** → Sistemas de comportamiento.
- **Data** → Perfiles y estructuras serializables.
- **Core** → Gestión global (GameManager, RoundSystem).

Desde el inicio he separado la captura de entrada de la lógica de movimiento para evitar dependencias innecesarias y mantener el código mantenible.

3. Prototipado del Escenario

He construido una arena 2D lateral coherente con la mecánica principal del juego.

El escenario incluye:

- Suelo base.
- Límites laterales.
- Plataformas a distintas alturas.
- Escalas ajustadas al tamaño del personaje.
- Sistema de colisiones funcional.

El diseño del escenario está pensado para favorecer el movimiento lateral y el salto entre plataformas, permitiendo testear correctamente las mecánicas físicas implementadas.

4. Controlador del Personaje (Runner)

El sistema de movimiento se ha dividido en dos partes claramente diferenciadas:

4.1 Captura de Entrada

Script: LocalRunnerInput

Este script se encarga exclusivamente de recoger:

- Movimiento horizontal.
- Salto (pulsado y mantenido).

No contiene lógica física ni comportamiento.

4.2 Sistema de Movimiento

Script: RunnerMovementSystem2D

En este sistema he implementado:

- Movimiento horizontal con aceleración y desaceleración.
- Salto con impulso físico.
- Coyote Time (permite saltar ligeramente después de abandonar el suelo).
- Jump Buffer (registra salto anticipado antes de tocar suelo).
- Salto variable según duración de la pulsación.
- Detección de suelo mediante BoxCast.
- Ajuste dinámico de gravedad para mejorar la sensación de caída.
- Posibilidad de bloqueo externo del movimiento (utilizado por DragSystem).

He utilizado Rigidbody2D para asegurar un comportamiento físico coherente y consistente con el motor.

5. Mecánica Asimétrica Inicial (Painter)

Se ha implementado una primera versión funcional del sistema de interacción del Painter:

- Arrastre temporal del Runner.
- Conversión temporal del Rigidbody a modo Kinematic.
- Aplicación de impulso físico al soltar.
- Sistema de cooldown proporcional con mínimo garantizado.
- Bloqueo temporal del control del Runner durante el arrastre.

Esta mecánica representa el núcleo asimétrico del proyecto y sirve como base para futuras habilidades parametrizadas.

6. Configuración Física

Se ha configurado el sistema físico con los siguientes parámetros:

- Gravity Y: -9.81
- Gravity Scale del Runner: 3
- Rotación bloqueada en eje Z.
- Collision Detection en modo Continuous.
- Interpolate activado para suavidad visual.

Las plataformas están asignadas a la Layer "Ground" para permitir una detección de suelo adecuada.

7. Estado Actual del Prototipo

En este punto el prototipo permite:

- Movimiento lateral fluido.
- Sistema de salto consistente.
- Interacción correcta con el entorno.
- Mecánica básica asimétrica funcional.
- Base estructural preparada para ampliación.

El objetivo de esta fase no ha sido ampliar mecánicas, sino asegurar que la estructura sobre la que se construirá el proyecto es estable.

8. Próximos Pasos

En las siguientes fases se desarrollará:

- Sistema de rondas.
- Sistema de vidas.
- AbilitySystem basado en perfiles serializados.
- Editor paramétrico de habilidades.
- Interfaz de usuario básica.

Conclusión

En esta primera entrega he conseguido:

- Plasmar la idea inicial en una escena jugable.
- Implementar un sistema de movimiento sólido.
- Introducir una mecánica asimétrica funcional.
- Establecer una arquitectura clara y escalable.

Como alumno que está comenzando a estructurar proyectos de mayor complejidad, he decidido priorizar organización, separación de responsabilidades y estabilidad antes que la expansión de funcionalidades.

Considero que esta base permitirá desarrollar el proyecto final con mayor coherencia técnica y menor riesgo de desorden estructural.